NASA Data Science & Astronomy Research Internship

Summer 2026

Exoplanet data analysis • NASA missions • Scientific writing

Dr. Peter Plavchan
George Mason University

landolt.gmu.edu | @PlavchanPeter
nasa.schar.program@gmail.com

Topics for our Program

Introductions

Logistics

How a NASA Mission Happens

Working with data, part 1; Jupyter notebooks and FITS files

Telescopes on the Ground and In Space

The NASA TESS mission and Transiting Exoplanets

Working with data, part 2; AstroImageJ

Making a Measurement – Science Methods and Inference

Science Writing

Working with data, part 3

Space Revolution and Space Policy

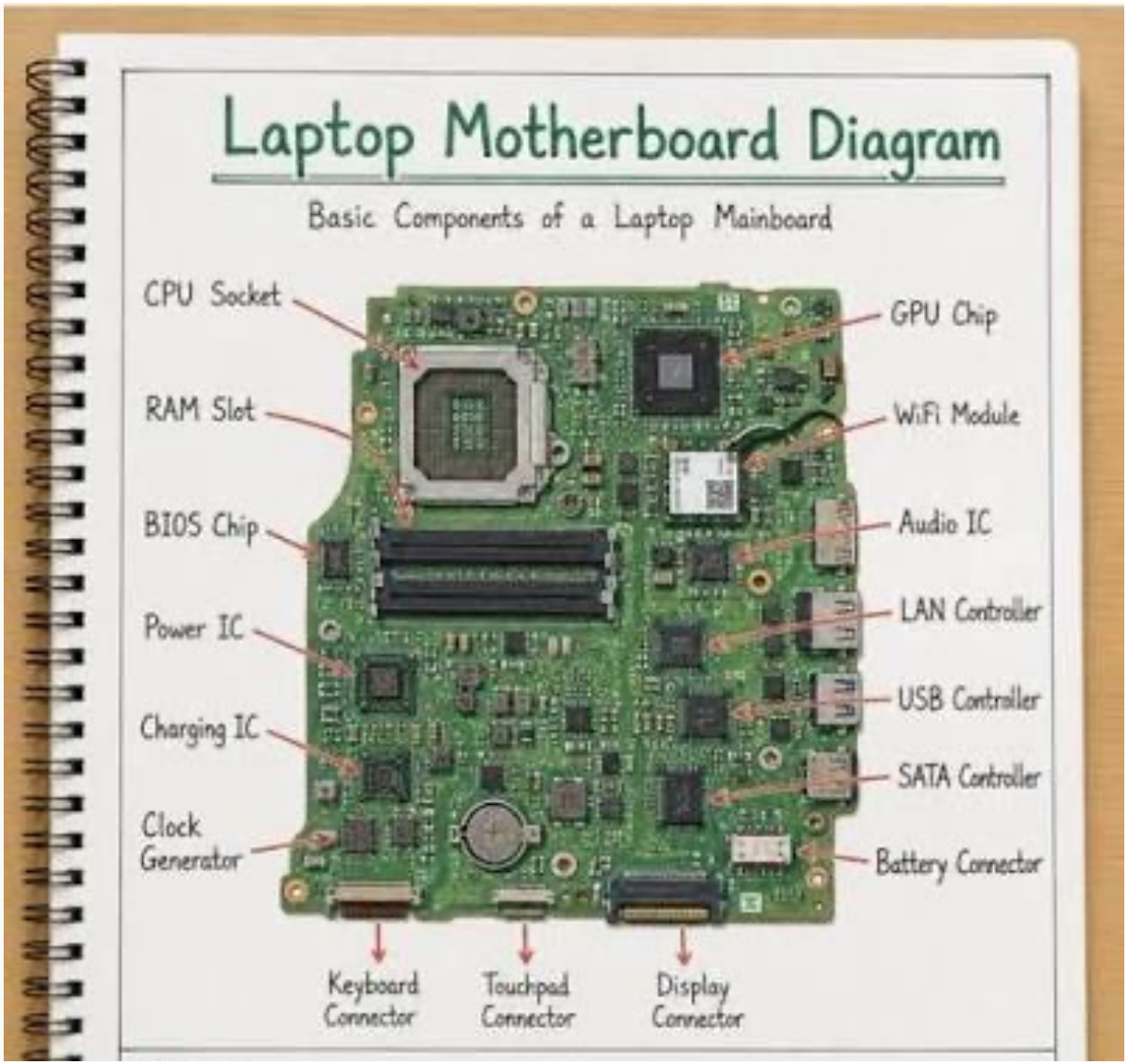
Closing Remarks

What are the main internal components of a computer?

Processor – executes a sequence of commands

Memory – stores information temporarily, such as a program and its contents

Hard drive – Stores information long-term, such as files and folders



What is a bit?

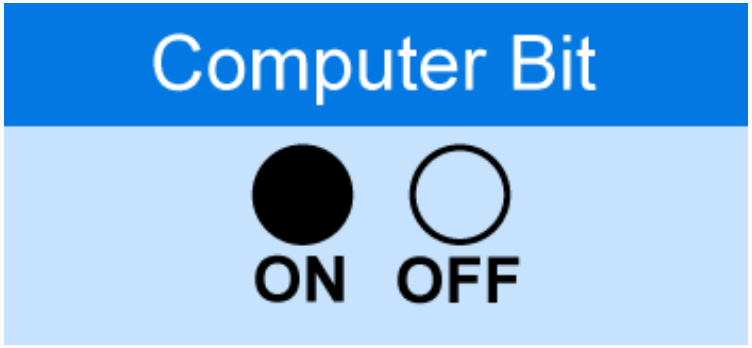
A bit is a 1 or 0 using binary logic

It can refer to the operation command a processor is executing. A 64-bit processor will take single commands as 64 bits at a time (and can have only 2^64 different commands).

It can refer to arrays or information stored in memory

It can refer to files or folders stored on a disk, such as a text file, or a compiled program

All characters and contents on a computer must be represented in bits.



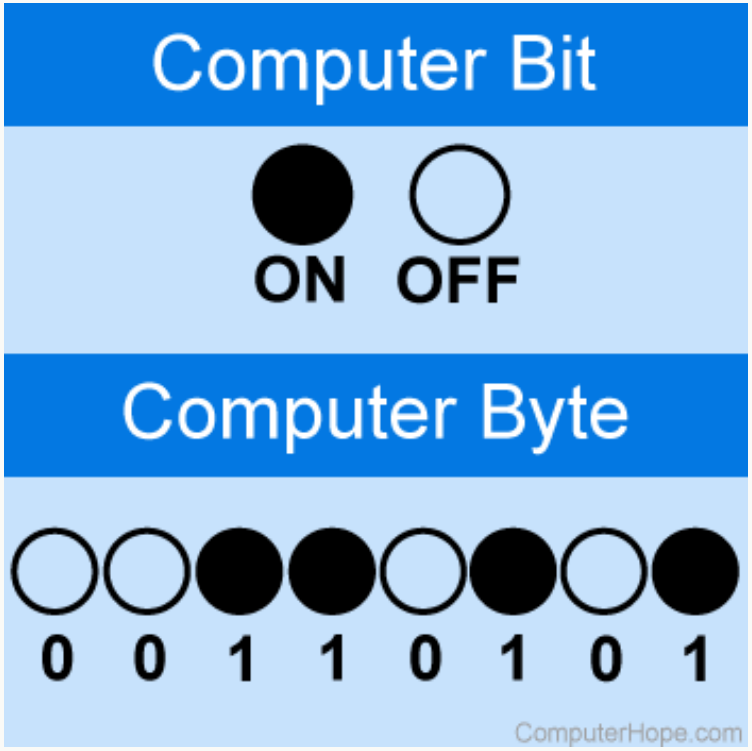
What is a byte?

A byte is 8 bits, e.g. 10101100

Bytes form the core of representing numbers and characters in memory and on hard disks, as well as commands that are executed by processors (and pixel value to be displayed on a monitor, and ...)

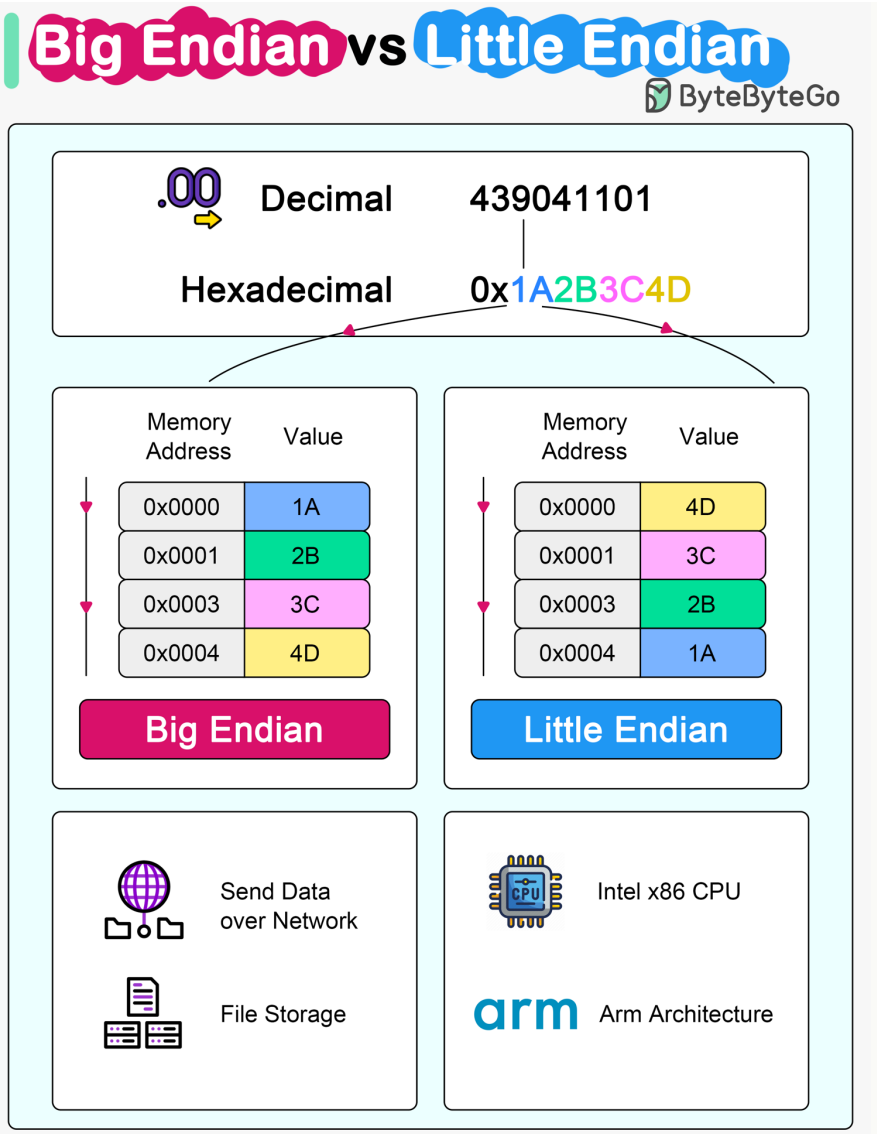
An integer number, like 25, can be represented in binary as 1101, but since computers use disk or memory space in whole bytes (unless compressed), the simplest representation of the number 25 is 00001101.

But even that isn't so simple, since that byte is what we call in programming an "unsigned short int", which can represent numbers of 0 to 255. What about negative numbers? Well, you can still do that with a single byte, but now you can only represent numbers -128 to +127, and this is called a "signed short int".



What about bigger numbers?

If you want to represent a number bigger than 255 or 128, you need an unsigned or signed int, which are two bytes each, and run from 0..65535 and -32768..+32767 respectively. What about bigger ints? Then you need long ints which are four bytes each (or 32 bits). What about bigger than that?



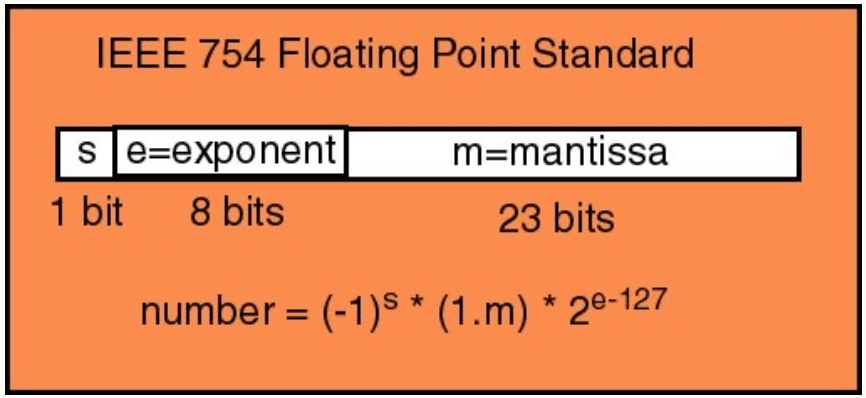
What about bigger numbers?

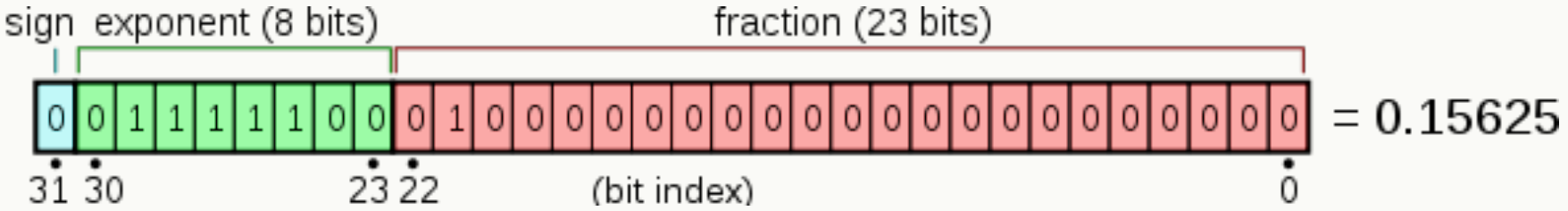
If you want to represent a number bigger than 255 or 128, you need an unsigned or signed int, which are two bytes each, and run from 0..65535 and -32768..+32767 respectively. What about bigger ints? Then you need long ints which are four bytes each (or 32 bits). What about bigger than that?

What about fractions or numbers after the decimal place?

Then things get more complicated, but still are still represented in bits and bytes. This is called a 32-bit float:

32-bit floats can have limited "machine precision", so there are "doubles" which are 64-bit floats, and even in some rarer scientific use cases even that amount of precision is not enough.

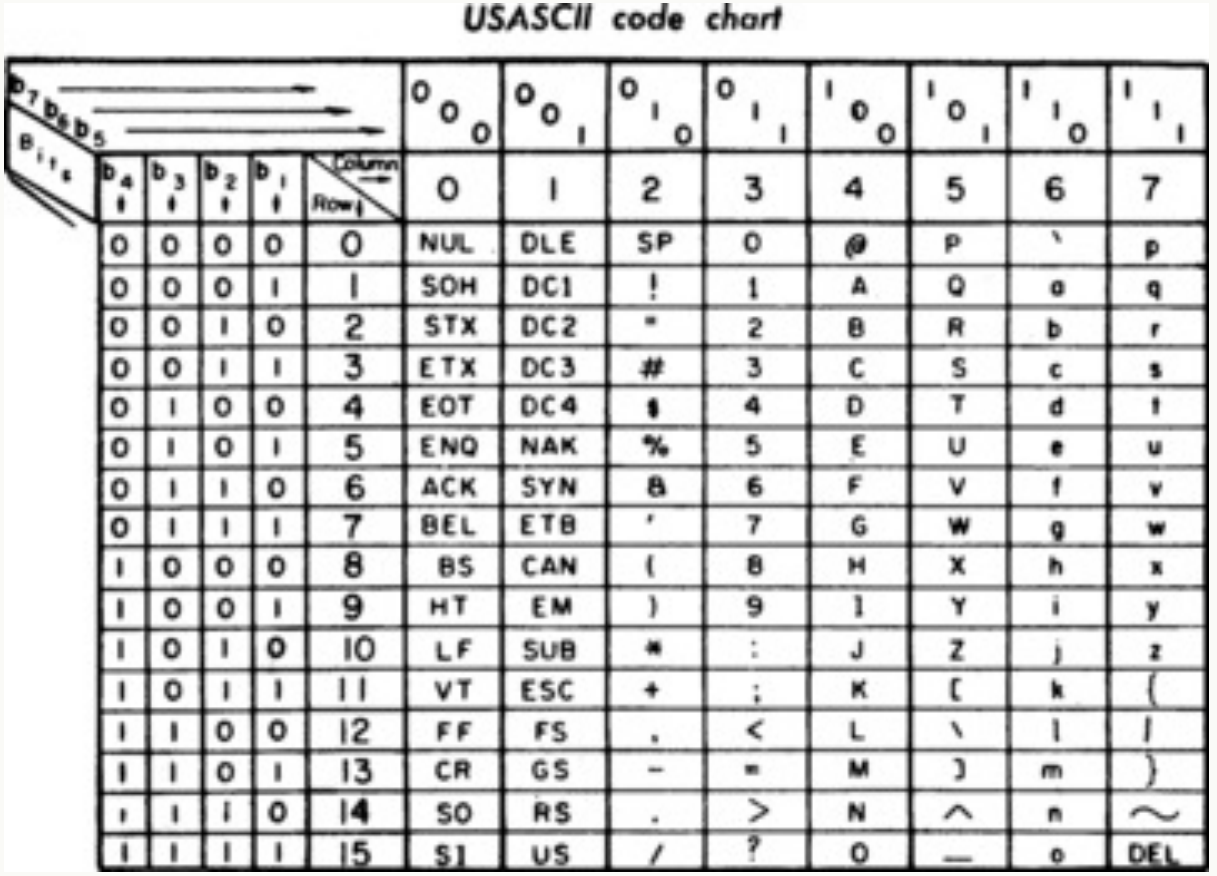




What about characters?

The first single byte of a character was known as ASCII.

Now multi-byte character standards are defined to cover more specialized characters and languages



USASCII code chart

Bits b_7 b_6 b_5 →	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b_4 b_3 b_2 b_1 ↓ Row ↓ Column →	0	1	2	3	4	5	6	7
0 0 0 0	0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1	1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0	2	STX	DC2	"	2	B	R	b	r
0 0 1 1	3	ETX	DC3	#	3	C	S	c	s
0 1 0 0	4	EOT	DC4	$	4	D	T	d	t
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v
0 1 1 1	7	BEL	ETB	'	7	G	W	g	w
1 0 0 0	8	BS	CAN	(	8	H	X	h	x
1 0 0 1	9	HT	EM	)	9	I	Y	i	y
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z
1 0 1 1	11	VT	ESC	+	;	K	[	k	{
1 1 0 0	12	FF	FS	,	<	L	\	l	|
1 1 0 1	13	CR	GS	-	=	M	]	m	}
1 1 1 0	14	SO	RS	.	>	N	^	n	~
1 1 1 1	15	SI	US	/	?	O	_	o	DEL

What about characters?

Computers often distinguish between binary and text files for determining how to display files with simple programs on the command line.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	Ç	160	A0	á	192	C0	└	224	E0	α
129	81	ü	161	A1	í	193	C1	┴	225	E1	ß
130	82	é	162	A2	ó	194	C2	┬	226	E2	Γ
131	83	â	163	A3	ú	195	C3	├	227	E3	π
132	84	ä	164	A4	ñ	196	C4	─	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	┼	229	E5	σ
134	86	å	166	A6	ª	198	C6	╞	230	E6	µ
135	87	ç	167	A7	º	199	C7	╟	231	E7	τ
136	88	ê	168	A8	¿	200	C8	╚	232	E8	Φ
137	89	ë	169	A9	⌐	201	C9	╔	233	E9	Θ
138	8A	è	170	AA	¬	202	CA	╩	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	╦	235	EB	δ
140	8C	î	172	AC	¼	204	CC	╠	236	EC	∞
141	8D	ì	173	AD	¡	205	CD	═	237	ED	ø
142	8E	Ä	174	AE	«	206	CE	╬	238	EE	ε
143	8F	Å	175	AF	»	207	CF	╧	239	EF	∩
144	90	É	176	B0	░	208	D0	╨	240	F0	≡
145	91	æ	177	B1	▒	209	D1	╤	241	F1	±
146	92	Æ	178	B2	▓	210	D2	╥	242	F2	≥
147	93	ô	179	B3	│	211	D3	╙	243	F3	≤
148	94	ö	180	B4	┤	212	D4	╘	244	F4	⌠
149	95	ò	181	B5	╡	213	D5	╒	245	F5	⌡
150	96	û	182	B6	╢	214	D6	╓	246	F6	÷
151	97	ù	183	B7	╖	215	D7	╫	247	F7	≈
152	98	ÿ	184	B8	╕	216	D8	╪	248	F8	°
153	99	Ö	185	B9	╣	217	D9	┘	249	F9	∙
154	9A	Ü	186	BA	║	218	DA	┌	250	FA	·
155	9B	¢	187	BB	╗	219	DB	█	251	FB	√
156	9C	£	188	BC	╝	220	DC	▄	252	FC	ⁿ
157	9D	¥	189	BD	╜	221	DD	▌	253	FD	²
158	9E	₧	190	BE	╛	222	DE	▐	254	FE	■
159	9F	ƒ	191	BF	┐	223	DF	▀	255	FF	

ComputerHope.com

Storing Data

Data formats are endless- xml, json, npz, pkl, csv, tsv, dat, sav, gz, txt, xls, xlsx, …

They are language specific, application specific, scientific field specific, and one can encounter lots of varieties in a single field.

You can invent your own!

The general principle of a header, a descriptive filename, and metadata, either stored in the data file itself, or a database associated with a set of files, applies to all of these formats.

Interestingly, writing and reading data files is very analogous to how computers talk to pieces of hardware – the bits are sent over and received from serial connections instead of to a file, with a format for interpreting the commands sent and received on either end.

Accessing Data

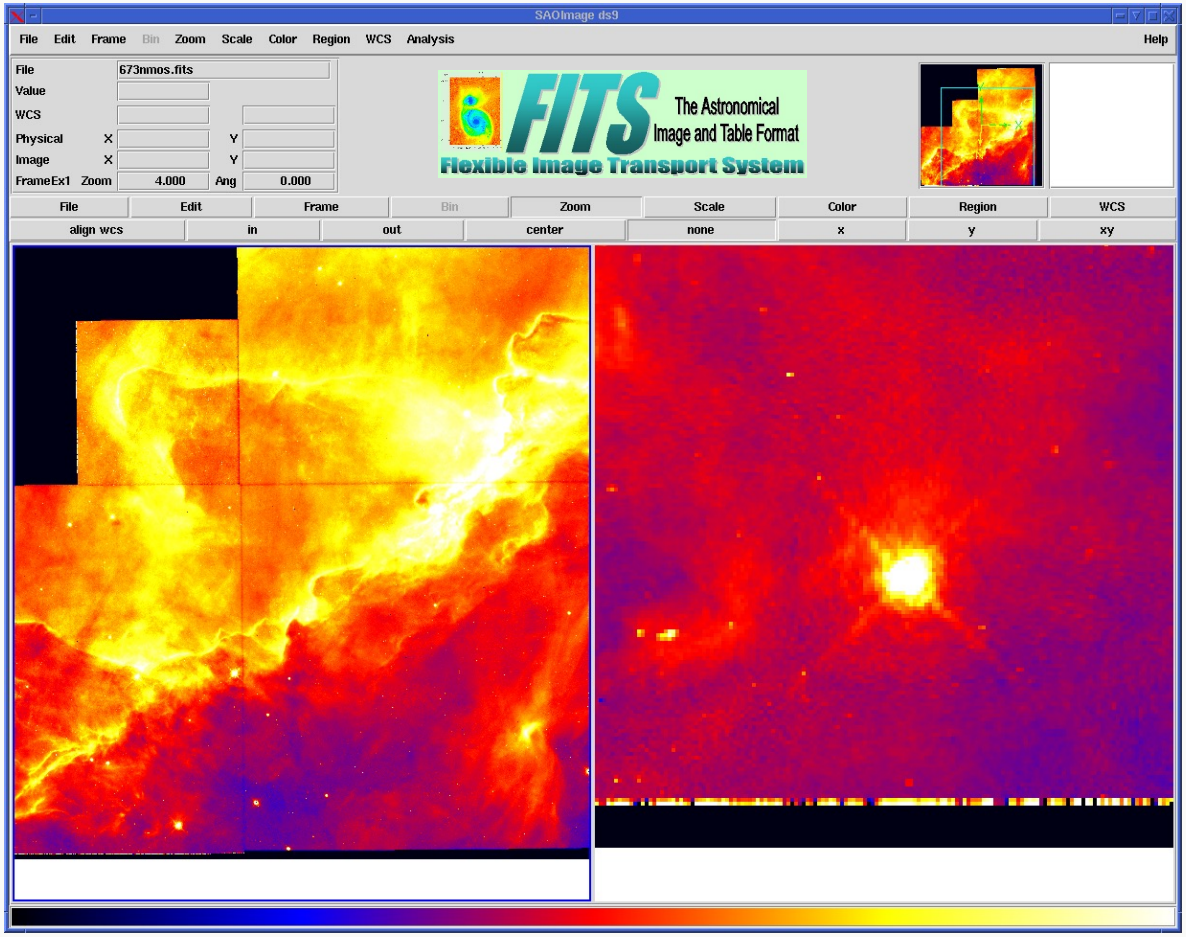
The tools are also endless for how data is accessed and manipulated.

Now let's work through one demo on accessing and manipulating data stored in FITS files for astronomy with a Jupyter notebook and python, which is not quite "low level", but is close to "low level", and we will see one more piece of software later – AstroImageJ

Generally, all tools, with a GUI or not, have to make some assumptions about the formatting of the data, and can range from a single grad student writing it, to a team of 50 professional developers working at a scientific organization.

OK, Now what about science data?

FITS file format



OK, Now what about science data?

FITS file format

Figure FITS-1

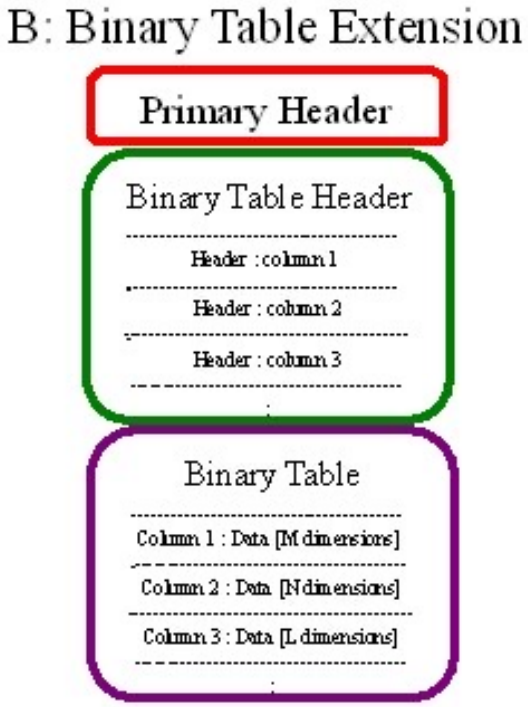
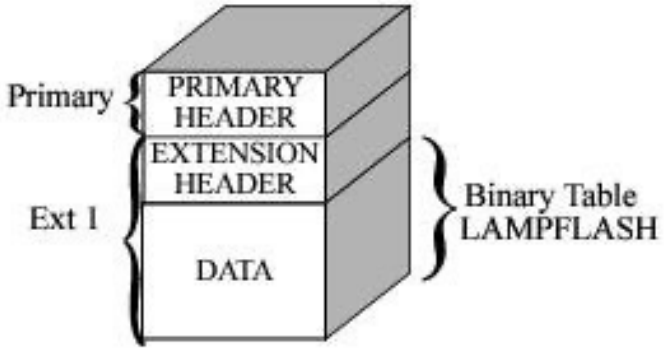
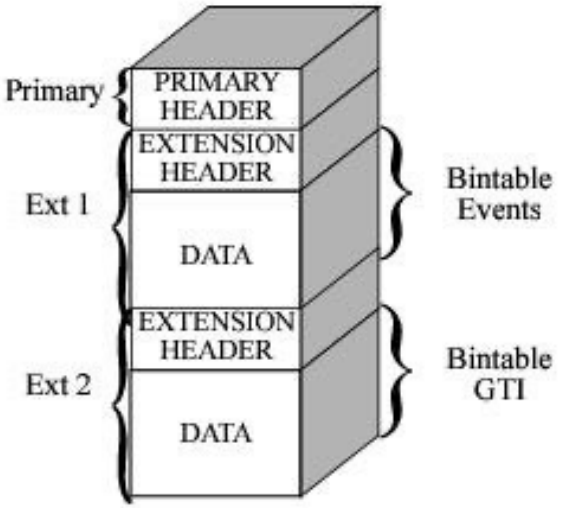


Figure FITS-2

OK, Now what about science data?

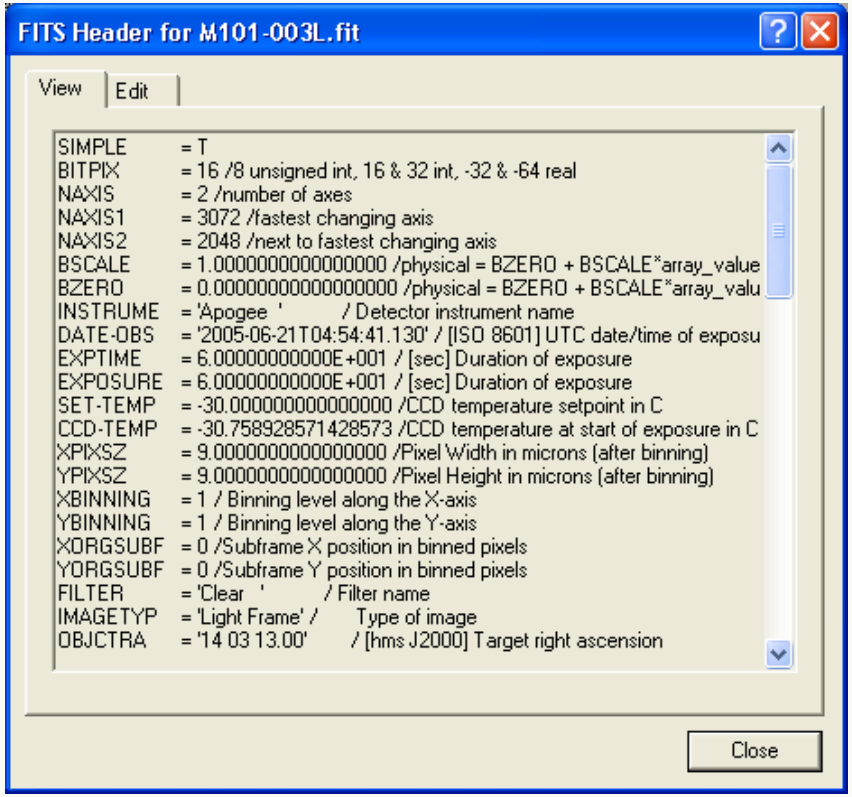
FITS file format

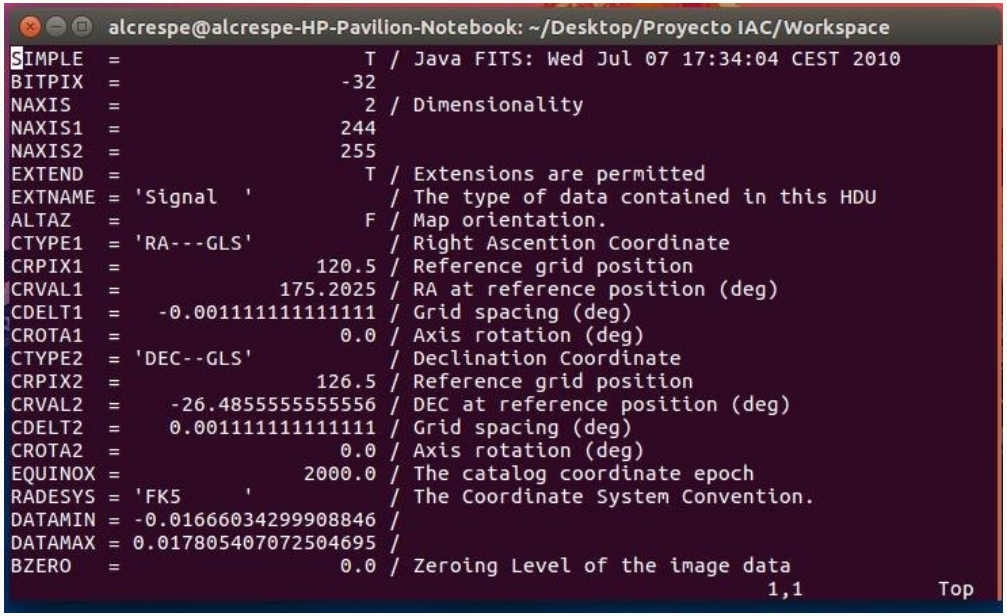




OK, Now what about science data?

FITS file format - https://en.wikipedia.org/wiki/FITS





Working with Data, Part 1

NASA Exoplanet Archive

MAST archive

Jupyter notebooks demo

Your assignment:

Download a TESS mission light curve for a TESS candidate exoplanet from the MAST archive

Use the Jupyter notebook to read in and plot the light curve of the TESS candidate exoplanet

Change the time and flux ranges of the plot to show one of the candidate planet transits

Post your plots to the Discord discussion channel

You will repeat this for your telescope observations candidate (when known) for you research paper.